

MechanicPro

Management System for a Mechanic Shop

System Architecture, Database Design, and Technical Overview

Author Eduardo Freitas Fernandes

Date May 2026

Document Technical Report

Version 1.2.0

Status **Final**

Language English

Abstract

This document is the technical report for **MechanicPro**, a desktop application designed to manage the day-to-day operations of a mechanic shop. The report describes the system architecture, the technology stack, the database design, and the build and release process.

MechanicPro is built with **Tauri v2** and **Rust** for the desktop shell, **React 19** and **TypeScript** for the user interface, and **SQLite** for local data persistence. The application runs entirely offline, with no server or cloud dependency.

The intended reader is a developer who wants to understand, maintain, or extend the system. Usage instructions are covered separately in the User Manual.

Keywords: Tauri, React, TypeScript, SQLite, Desktop Application, Mechanic Shop Management.

Revision History

Version	Date	Description	Author
1.0.0	Jan 2026	Initial document created	Your Name
1.1.0	Mar 2026	Architecture section expanded	Your Name
1.2.0	May 2026	Database and build sections added	Your Name

This document describes version 1.2.0 of MechanicPro. For changes to the application itself, refer to the project `CHANGELOG.md`.

Contents

1	Introduction	6
1.1	Purpose	6
1.2	Scope	6
1.3	Intended Audience	6
1.4	Document Conventions	6
2	System Overview	7
2.1	Application Purpose	7
2.2	Technology Stack	7
2.3	System Requirements	7
2.4	High-Level Data Flow	7
3	Architecture	8
3.1	Overview	8
3.2	Frontend — React and TypeScript	8
3.2.1	Component Structure	8
3.2.2	State Management	8
3.2.3	Routing	8
3.3	Backend — Tauri and Rust	8
3.3.1	Tauri Commands	8
3.3.2	Plugin Usage	8
3.4	IPC Communication	8
3.5	Folder Structure	8
4	Database	9
4.1	Overview	9
4.2	Schema	9
4.2.1	clients	9
4.3	Entity-Relationship Diagram	9
4.4	Business Rules	9
4.5	CSV Format	9
5	Build and Release	10
5.1	Development Environment Setup	10
5.2	Build Process	10
5.3	Output Artefacts	10
5.4	Configuration	10
6	Known Limitations	11
A	Glossary	12
B	References	13

List of Figures

List of Tables

1 Introduction

1.1 Purpose

1.2 Scope

1.3 Intended Audience

1.4 Document Conventions

Note: Blue boxes like this one highlight important information or warnings throughout the document.

2 System Overview

2.1 Application Purpose

2.2 Technology Stack

Layer	Technology	Role
UI Framework	React 19 + TypeScript	Frontend interface
Build Tool	Vite	Dev server and bundler
Desktop Shell	Tauri v2 (Rust)	Native window and OS access
Database	SQLite (via Tauri SQL plugin)	Local data persistence
Routing	react-router-dom	Page navigation
Charts	recharts	Dashboard visualisations
PDF Generation	jsPDF + html2canvas	Service document export
Icons	lucide-react	UI icon set

2.3 System Requirements

2.4 High-Level Data Flow

3 Architecture

3.1 Overview

3.2 Frontend — React and TypeScript

3.2.1 Component Structure

3.2.2 State Management

3.2.3 Routing

3.3 Backend — Tauri and Rust

3.3.1 Tauri Commands

3.3.2 Plugin Usage

3.4 IPC Communication

3.5 Folder Structure

Listing 3.1: Repository structure

```
1  mechanic_shop/  
2  +--- src/                                # React + TypeScript source  
3  |   +--- components/                    # Reusable UI components  
4  |   +--- pages/                        # Route-level page components  
5  |   '--- ...  
6  +--- src-tauri/                         # Rust / Tauri backend  
7  |   +--- src/  
8  |   |   +--- main.rs                   # Entry point  
9  |   |   '--- ...  
10 |   '--- tauri.conf.json  
11 +--- public/                            # Static assets  
12 +--- package.json  
13 '--- vite.config.ts
```

4 Database

4.1 Overview

4.2 Schema

4.2.1 clients

Column	Type	Constraints	Description
id	INTEGER	PK, AUTOINCREMENT	Unique client identifier
name	TEXT	NOT NULL	Full name of the client
phone	TEXT		Contact phone number
email	TEXT		Contact email address
created_at	TEXT	NOT NULL	ISO 8601 creation timestamp

4.3 Entity-Relationship Diagram

4.4 Business Rules

4.5 CSV Format

Column	Type	Notes
--------	------	-------

5 Build and Release

5.1 Development Environment Setup

Listing 5.1: Install dependencies and start dev server

```
1 npm install
2 npm run tauri dev
```

5.2 Build Process

Listing 5.2: Production build

```
1 npm run tauri build
```

5.3 Output Artefacts

5.4 Configuration

6 Known Limitations

A Glossary

B References

- Tauri Documentation: <https://tauri.app/>
- React Documentation: <https://react.dev/>
- SQLite Documentation: <https://www.sqlite.org/docs.html>